LOURDES MATHA COLLEGE OF SCIENCE & TECHNOLOGY
LOURDES HILLS, KUTTICHAL P. O., THIRUVANANTHAPURAMM - 695 574

29/30

ANSWER BOOK

Name DEEPAK.V Class No 17 Branch MCA (2021-23)

Subject Design & Analysis of Algorithms Semester S3

Mark Tabulation

Mark Divisions	Q.1	Q.2	Q.3	Q.4	Q.5	Q.6	Q.7	Q.8	Q.9	Q.10	Q.11	Q.12	Q.13	Q.14	Q.15
	3	3	3	3	2	3	6	6							
Total															
Grand Total													29		

8. Given,

$(P_1, P_2, P_3) = (500, 900, 700)$ $m = 5$

$(w_1, w_2, w_3) = (1, 3, 7)$ ~~$x = \{0, 0, 0\}$~~

Initially,

Items	1	2	3
Profit	500	900	700
Weight	1	3	7
P_i/w_i	500	300	100

$x = \{0, 0, 0\}$

x, be solution vector.

The Items need to be arranged in decreasing

Since, Here all [illegible] step.
order So, no need

Initially step-1

$m' = 5$ ($m' = m$)

Item 1 profit = 500

weight = $1 < 5$

So, $x_i = 1$

Now, $x = \{1, 0, 0\}$

need weight, $m' = m' - w[i]$

$m' = 5 - 1$

$m' = 4$

Step-2

Item 2 profit = 900

weight = $3 < 4$

So, $x_i = 1$

Now, $x = \{1, 1, 0\}$

need weight, $m' = m' - w[i]$

Item 3

Profit = 700

weight = 7 > 1

So, reduce the weight upto the current knapsack capacity i.e. m = 1

we know

$w_i x_i \leq m$

$7 x_i \leq 1$

$x_i \leq \frac{1}{7}$

So, profit = $700 \times \frac{1}{7} = 100$

weight = $7 \times \frac{1}{7} = 1$

$x = \{1, 1, 0.14\}$

for the items given the knapsack having capacity 5 can be filled on maximum profit according to the below solution matrix

$$x = \{1, 1, 0.14\}$$

The maximum profit obtained is.

$$\max \sum_{i=0}^{2} p_i x_i \Rightarrow 500 + 900 + 100$$

$$= 1500$$

Algorithm KnapSack (m, n)

{

/* p[1:n] and w[1:n] be
given items respectively profit and weights of
the order p[i]/w[i] > p[i+1]/w[i+1] */

/* m be the knapsack capacity and
x[1:n] be the solution vector in which each
element is of range $0 \le x_i \le 1$ */

for i=1 to n

{ x[i] = 0.0

U = m

for i=1 to n

{

if (w[i] > U) then break;

x[i] = 1.0

U = U - w[i]

}

if (i < n) then x[i] = U/w[i]

}

7. Quick Sort

Algorithm:

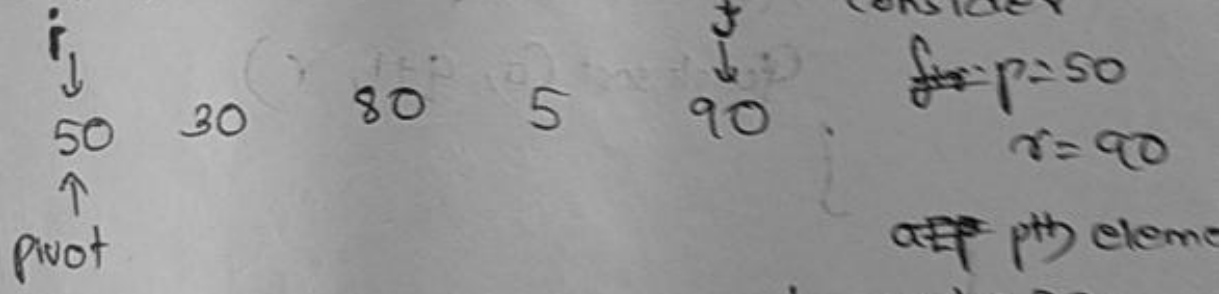
```
Algorithm QuickSort(a, p, r)
{
    if (p < r)
    {
        q = partition(a, p, r)
        Quicksort(a, p, q-1)
        Quicksort(a, q+1, r)
    }
}

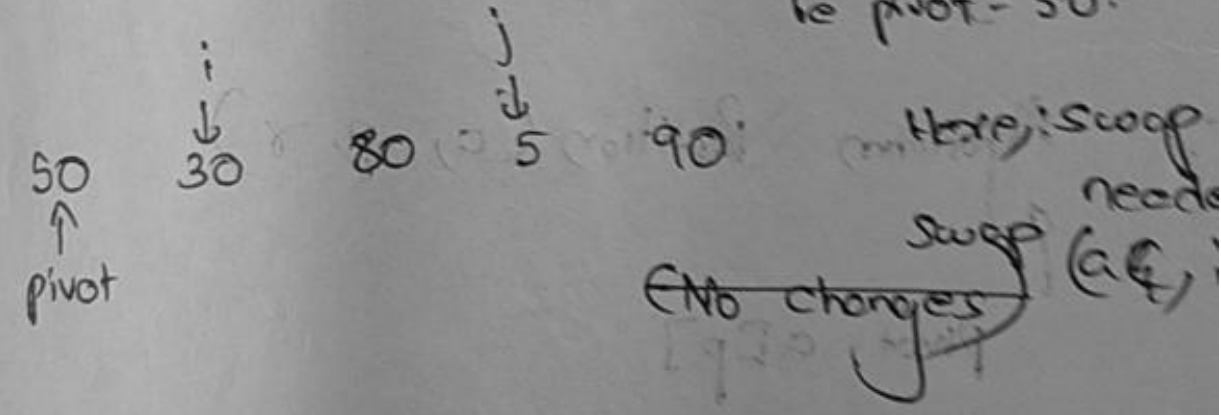
Algorithm Partition(a, p, r)
{
    pivot = a[p]
    i = p-1, j = r-1
    while TRUE
    {
        do
```

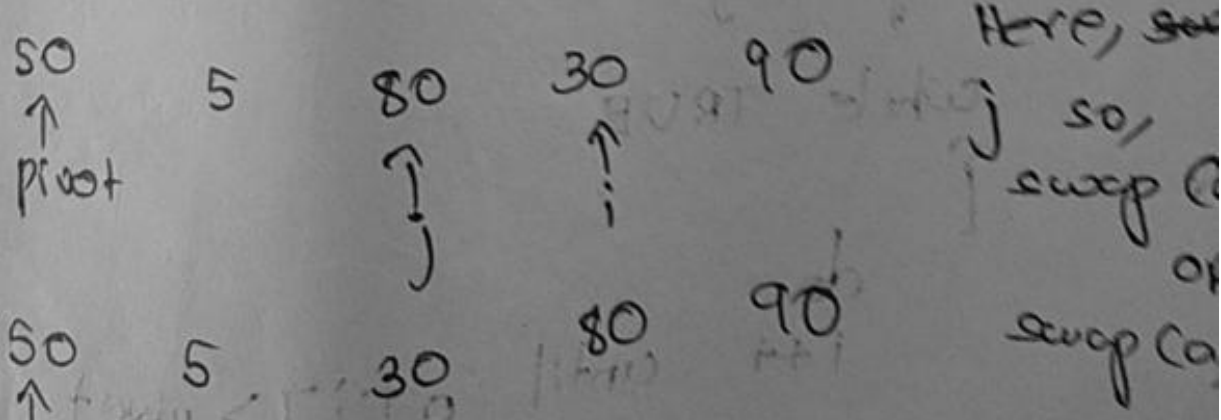
```
    if (i < j)
        swap(a, i, j)
    }
    swap(a, j, p)
    return j
}
```

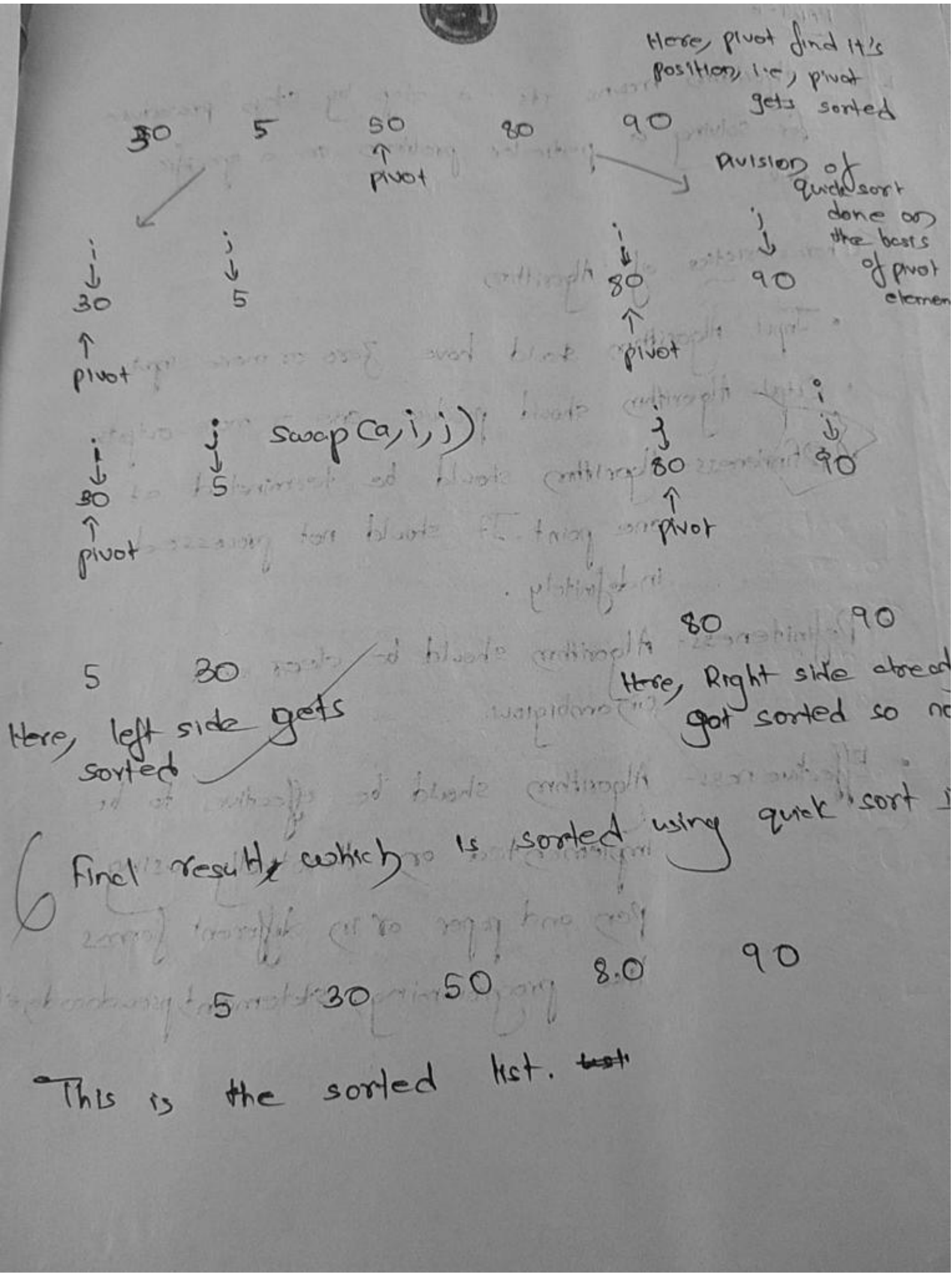
Q. Problem

50, 36, 80, 5, 90








Here, pivot find it's position, i.e, pivot gets sorted
30 5 50 80 90
pivot
Division of quicksort done on the basis of pivot element
i j
30 5
pivot
i j
80 90
pivot
swap(a,i,j)
i j
30 5
pivot
i j
80 90
pivot
5 30
Here, left side gets sorted
80 90
Here, Right side already got sorted so no
Final result which is sorted using quick sort
5 30 50 80 90
This is the sorted list.

1. • Algorithm
for solving a particular problem or a specific task.

Characteristics of Algorithm

- Input - Algorithm should have zero or more inputs
- Output - Algorithm should produce one or more outputs.
- Finiteness - Algorithm should be terminated at one point. It should not processed indefinitely.
- Definiteness - Algorithm should be clear and unambigious.
- Effectiveness - Algorithm should be effective to be implemented or theoretically using pen and paper or in different form like programming, statement, pseudo

Time Complexity

1.

- Time Complexity means the amount of time needed to execute a particular algorithm.
- Time Complexity = Compile Time + Run Time
- There are many methods for Computing Time Complexity. The most common method is step count or frequency count
  - Constants, ~~are~~ are cont counted as 1 unit
  - Comments as 0 unit
  - loops as n+1 unit
- The aim of time complexity is to evalu the performance of an algorithm or the basis of the execution time as a whole. which is desired to be minimum

Example,

```
Algorithm sum-of-array (a, n)
{                                          ——— 1
    s=0
    for i=1 to n                           ——— n+1
        s=s+a[i]                           ——— 1
    return s                               ——— 1
}
```

Here Time Complexity is $n+3$ (4).

Space Complexity.

- Space Complexity means the memory space taken by the algorithm to perform it's execution.
- The aim of space complexity to evaluate the performance of an algorithm on the basis of memory space consumption which is desired to be minimal.

- The most common way of calculating the space complexity is to count the unit of allotted memory. Having formula,

$$S_p = c + S_p(I)$$

where, S_p = Space Complexity

c = Constant

$S_p(I)$ = ~~Inst~~ Variable at ~~the~~ particular instance.

- So Space complexity is the sum of content and instance variable.

Example;

```
Algorithm sum_of_array(a, n)
{
    s = 0
    for i = 1 to n
        s = s + a[i]
    return s
}
```

n = 1
s = 1
i = 1
} Total 3

then,

array, a = n

Total = n + 3

S_p: ~~A+B~~ $n+3$

3. Control Abstraction for Divide and Conquer.

```
Algorithm DkC (P)
{
    if SMALL (P) then S(P)
    else
    {
        Divide P into k instances where,
        P1, P2, P3 : : : Pk
        where, k≥1
    }
    return DkC(P1), DkC(P2), DkC(P3) . . . D
}
```

Here, SMALL (P) is Boolean which returns True which problem P is so small to be solved. Else False.

S(P) means solution to the given problem

4. Control Abstraction of Greedy strategy

```
Algorithm Greedy (a, n)
{
    solution = ϕ        // solution vector.
    ~~if(feasible~~
    for i = 1 to n
    {
        x = select (a, n)   // select an solution
                            based on given constraints
        if (Feasible (solution, x))
            then
                solution = union (solution, x)
                // If the solution which is sele
                is feasible or better from
                solution then add it to
                solution vector.
        else
            reject   // Reject the solution
    }
    return solution   // return solution vector o
}
```

3. Minimum Spanning Tree is the Graph having structure which is obtained from the graph in which,

- Spanning Tree
- All vertices are visited or traversed edges
- Cycles are removed i.e, cyclic edges or graph
- It does have some cost to traverse from one vertex to another. Thus, minimum spanning tree is obtained from weighted graph and output as weighted tree.

- If The Spanning Tree is the optimum to use of minimum. i.e, generates only minimum cost from all the possible combinations available for particular weighted graph is known as minimum cost spanning Tree.

Minimum Cost spanning Tree is obtained using Greedy Algorithm approach using two famous algorithms.

→ Kruskal's Algorithm

- Minimum cost spanning tree mathematically formed as

If, $G = (V, E)$

then, ~~MST = min {G(V,~~

$$MST = \min \left\{ \sum_{e_i} x_i : x \in G(V, E') \right\}$$

6. Job Sequencing problem

- Job Sequencing problem is one of the application problem ~~does~~ which is solved using Greedy Algorithm Approach.
- In ~~the~~ which certain no: of jobs are provid and it's corresponding time for completing these jobs are being given.
- The task is to obtain a combination set in such a way that, In the given deadline, The jobs need to be done & on maximum profit & gain.
- So, This problem can also be stated a maximization problem on the basis of pro

- This problem ... as Job sequencing with deadline.

Algorithm:

Algorithm JobSequencing (d, J, n)

{

/* Jobs J are given with corresponding time limit times to be completed and having deadline of d to complete task with maximum profit under or within deadline d */

J = {1}

for i = 2 to n

{

if (J ∪ {i} jobs can be completed within deadline d)

then

J = J ∪ {i}

}

}